



A+ LAB SERIES V5

Lab 9: Linux Users and Groups

Document Version: **2026-01-02**

Material in this Lab Aligns to the Following	
CompTIA A+ (220-1201) Exam Objectives	
CompTIA A+ (220-1202) Exam Objectives	1.9: Identify common features and tools of the Linux client/desktop operating system. 2.7: Given a scenario, apply workstation security options and hardening techniques.

Contents

Introduction	3
Objective	3
Lab Topology	4
Lab Settings	5
1 Understanding File Permissions.....	6
1.1 Learn to Read and Interpret File Permissions.....	6
2 Changing File Ownership	10
2.1 Linux Commands to Change Ownership.....	10
3 Modify File Permissions	14
3.1 Linux Commands to Modify Permissions.....	14
3.2 Recursive Permission Changes.....	14

Introduction

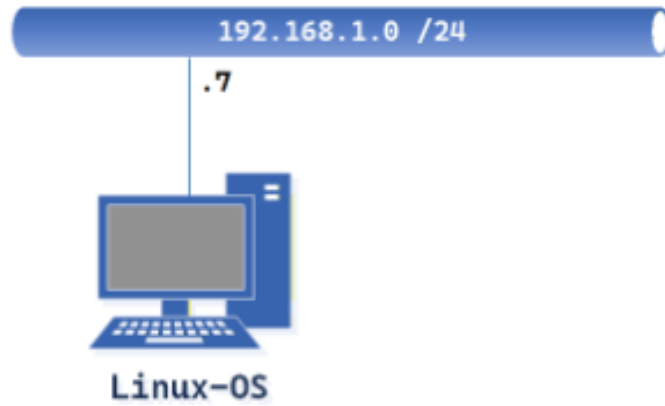
This lab will introduce you to the basics of user and group permissions in Linux Mint. You will learn how to interpret file permissions and use the `chown` and `chmod` commands to manage access to files and directories. By the end of this lab, you will have a fundamental understanding of how to control and modify permissions in a Linux environment.

Objective

This lab provides hands-on experience with file permissions in Linux Mint, which is crucial for understanding and managing system security effectively. It also reviews key concepts related to Linux user and group management. In this lab, we will be performing the following tasks:

- Understand File Permissions:
 - Learn to read and interpret the `rwx-----` file permission notation.
- Change File Ownership:
 - Use the `chown` command to change the owner and group of files and directories.
- Modify File Permissions:
 - Use the `chmod` command to modify file and directory permissions.

Lab Topology



Lab Settings

The information in the table below will be needed in order to complete the lab. The task sections below provide details on the use of this information.

Virtual Machine	IP Address	Account (if needed)	Password (if needed)
Linux-OS	192.168.1.7 / 24	sysadmin	NDGlabpass123!

1 Understanding File Permissions

File permissions in Linux Mint 21 (Vanessa) are a critical aspect of system security and user management, delineating access rights for files and directories through a structured permission system. This system categorizes users into three types—owner, group, and others—and assigns permissions for read (r), write (w), and execute (x) operations. Permissions are represented either in a string format, such as `rw-r--r--`, indicating owner, group, and others' permissions respectively, or numerically, where each permission bit corresponds to a value (read=4, write=2, execute=1), summing to a total permission value (e.g., `rw-r--r--` = 7). Understanding these permissions is essential for maintaining system integrity by controlling who can view, modify, or execute files and directories. For instance, a permission set of `-rw-r--r--` allows the owner to read and write, while others can only read the file. Effective management of these permissions through commands like `chmod` to change permissions and `chown` to change file ownership ensures that access is appropriately restricted or granted, adhering to the principle of least privilege and thereby enhancing system security and operational efficiency.

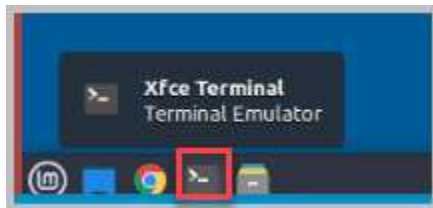
1.1 Learn to Read and Interpret File Permissions

1. Open a console connection to the **Linux-OS** system.

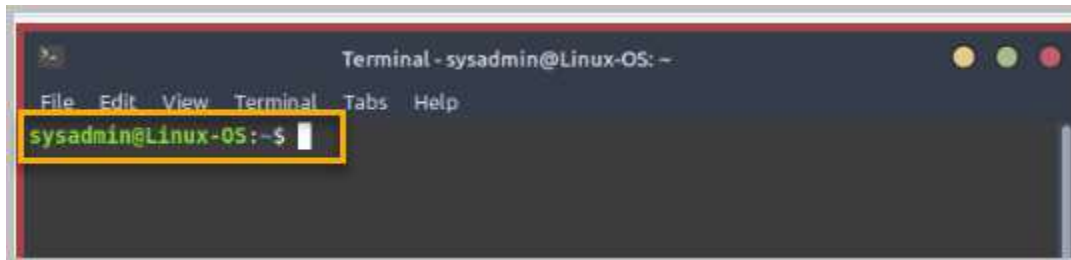


To launch the console for a lab device, you can click on the respective tab in the navigation bar at the top or navigate to the *Topology* section and click the lab device's corresponding image.

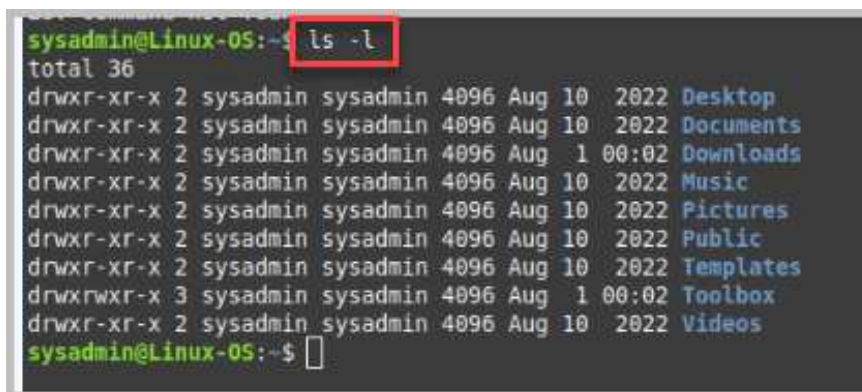
- Click the **XFce Terminal Emulator** icon on the taskbar.



- Notice you are currently logged into *sysadmin* on the *Linux-OS* system.



- Type the command `ls -l`. The `ls -l` command lists directory contents in a detailed format, providing comprehensive information about files and directories.





When you run `ls -l`, each line of the output represents a file or directory and contains the following elements:

1. **File Type and Permissions** (first column):
 - **First Character:** Indicates the file type:
 - d for directory
 - - for regular file
 - l for symbolic link
 - c for character device
 - b for block device
 - p for FIFO (named pipe)
 - s for socket
 - **Next Nine Characters:** Represent permissions in three sets of three for owner, group, and others:
 - r (read), w (write), x (execute), or - (no permission)
2. **Number of Hard Links** (second column):
 - Indicates how many hard links are associated with the file. For directories, this number includes `.` and `...`
3. **Owner** (third column):
 - The username of the file's owner.
4. **Group** (fourth column):
 - The group name to which the file belongs.
5. **Size** (fifth column):
 - The size of the file in bytes.
6. **Date and Time** (sixth and seventh columns):
 - When the file was last modified. If the file was modified within the last six months, it shows the date and time. Otherwise, it shows the year.
7. **Filename** (last column):
 - The name of the file or directory.

5. Understanding the `ls -l` command output is fundamental for managing files and directories effectively in Linux, ensuring proper access control and system organization.

```
sysadmin@Linux-05:~$ ls -l
total 36
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Desktop
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Documents
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 1 00:02 Downloads
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Music
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Pictures
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Public
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Templates
drwxrwxr-x 3 sysadmin sysadmin 4096 Aug 1 00:02 Toolbox
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Videos
sysadmin@Linux-05:~$
```

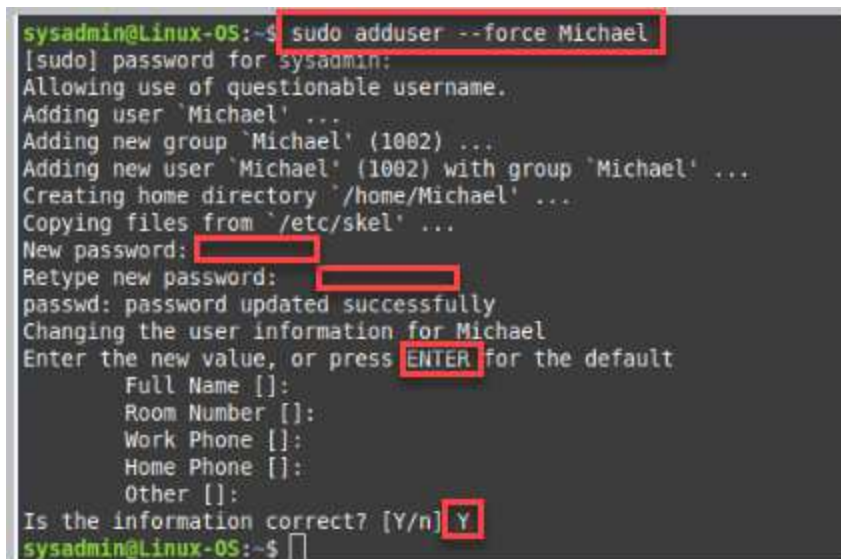


- d indicates it's a directory.
- rwxr-xr-x shows permissions: owner has read, write, execute; group and others have read and execute.
- 2 hard links.
- sysadmin is the owner, sysadmin is the group.
- 4096 bytes in size
- Modified date
- Directory Name

2 Changing File Ownership

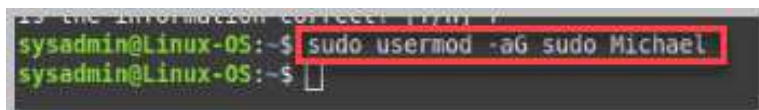
2.1 Linux Commands to Change Ownership

1. Create a new user for the *Linux-OS* system. Type the command `sudo adduser --force Michael`. Enter the password for *sysadmin* `NDGlabpass123!` and hit **Enter** on your keyboard. The user account *Michael* will be created and added to the *User* Group. Enter new password `NDGlabpass123!` For Michael's account, hit **enter** to accept defaults for *Full Name*, *Room Number*, *Work Phone*, *Home Phone*, and *Other*. Type **Y** verifying the information is correct.



```
sysadmin@Linux-OS:~$ sudo adduser --force Michael
[sudo] password for sysadmin:
Allowing use of questionable username.
Adding user 'Michael' ...
Adding new group 'Michael' (1002) ...
Adding new user 'Michael' (1002) with group 'Michael' ...
Creating home directory '/home/Michael' ...
Copying files from '/etc/skel' ...
New password: 
Retype new password: 
passwd: password updated successfully
Changing the user information for Michael
Enter the new value, or press ENTER for the default
  Full Name []: 
  Room Number []: 
  Work Phone []: 
  Home Phone []: 
  Other []: 
Is the information correct? [Y/n] Y
sysadmin@Linux-OS:~$
```

2. Add *Michael's* account to the *Administrator Group* by typing the command `sudo usermod -aG sudo Michael`.



```
sysadmin@Linux-OS:~$ sudo usermod -aG sudo Michael
sysadmin@Linux-OS:~$
```



sudo, which stands for "Superuser Do," is a command in Linux that allows users to execute commands with elevated privileges, typically those of the root user, by entering their password. This mechanism enhances system security by reducing the need for users to log in as root, thereby minimizing the risk of accidental system modifications or security breaches. The behavior of *sudo* is controlled by the */etc/sudoers* file, which dictates who can use *sudo*, what commands they can run, and on which systems. When a user precedes a command with *sudo*, they are prompted for their password, and upon successful authentication, the command executes with superuser privileges. *sudo* also features logging capabilities, recording all commands run with elevated permissions in system logs like */var/log/auth.log*, providing an essential audit trail. Additionally, *sudo* includes features like session timeouts, requiring re-authentication after inactivity, and the ability to run commands as different users or with specific restrictions based on *sudoers* configurations. This command is pivotal for system administration, allowing controlled access to system-altering commands while maintaining a secure environment.

3. Type the command `touch file1.txt file2.txt`. This command creates 2 text files in the *sysadmin*'s home directory.



```
sysadmin@Linux-OS:~$ touch file1.txt file2.txt
sysadmin@Linux-OS:~$
```



The *touch* command in Linux serves as a versatile tool for both creating new, empty files and updating the timestamps of existing files or directories. When you run *touch filename*, it will create a new file named *filename* if it does not already exist, or if it does, it updates the file's access and modification times to the current system time without altering its content. This command can also handle multiple files simultaneously, making it efficient for batch operations. Additionally, *touch* can modify only the access or modification times separately with options like *-a* for access time or *-m* for modification time, and it supports setting custom timestamps with the *-d* option. Its simplicity and effectiveness make *touch* an essential command for tasks ranging from file creation in scripts to timestamp manipulation for testing or maintenance purposes.

4. Type the command `ls -l`. The `ls -l` command lists directory contents in a detailed format, providing comprehensive information about files and directories. Notice the two text files you created using the `touch` command and the ownership of both files.

```
sysadmin@Linux-OS:~$ ls -l
total 36
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Desktop
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Documents
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 1 00:02 Downloads
-rw-rw-r-- 1 sysadmin sysadmin 0 Nov 1 12:53 file1.txt
-rw-rw-r-- 1 sysadmin sysadmin 0 Nov 1 12:53 file2.txt
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Music
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Pictures
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Public
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Templates
drwxrwxr-x 3 sysadmin sysadmin 4096 Aug 1 00:02 Toolbox
drwxr-xr-x 2 sysadmin sysadmin 4096 Aug 10 2022 Videos
sysadmin@Linux-OS:~$
```

5. Type the command `sudo chown Michael file1.txt` to change the owner of `file1.txt`.

```
sysadmin@Linux-OS:~$ sudo chown Michael file1.txt
sysadmin@Linux-OS:~$
```

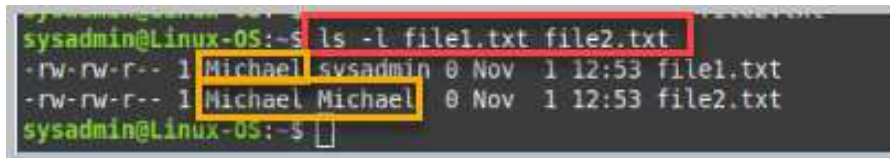
6. Type the command `sudo chown Michael:Michael file2.txt` to change both the owner and group for `file2.txt`. This command also creates a new group called `Michael`.

```
sysadmin@Linux-OS:~$ sudo chown Michael:Michael file2.txt
sysadmin@Linux-OS:~$
```



The `chown` command in Linux is used to change the ownership of files and directories. It allows you to transfer ownership from one user or group to another, which is crucial for managing access rights and permissions on a system. When you run `chown`, you specify the new owner, optionally followed by a colon, and the new group, followed by the file or directory you wish to change. For instance, `chown newuser:newgroup filename` would change both the owner and group of `filename` to `newuser` and `newgroup`, respectively. If you only specify a username without a group, the file's group will remain unchanged. Conversely, using `--` followed by a group name changes only the group ownership. `chown` can also operate recursively on directories with the `-R` option, ensuring that all files and subdirectories within the specified directory also have their ownership changed. This command is essential for system administrators to manage user access, ensure proper permissions, and maintain system security by controlling who can manipulate or access specific files and directories.

7. Verify the changes for files *file1.txt* and *file2.txt* by typing command `ls -l file1.txt file2.txt`.



```
sysadmin@Linux-05:~$ ls -l file1.txt file2.txt
-rw-rw-r-- 1 Michael sysadmin 0 Nov  1 12:53 file1.txt
-rw-rw-r-- 1 Michael Michael  0 Nov  1 12:53 file2.txt
sysadmin@Linux-05:~$
```

3 Modify File Permissions

In Linux, file permissions can be modified using the `chmod` command, which adjusts read, write, and execute permissions for the file's owner, group, and others.

3.1 Linux Commands to Modify Permissions

1. Type command `sudo chmod u+x file1.txt` to make `file1.txt` executable for the file owner.

```
sysadmin@Linux-OS:~$ sudo chmod u+x file1.txt
[sudo] password for sysadmin:
sysadmin@Linux-OS:~$
```

2. Type the command `sudo chmod go-w file2.txt` to remove write permissions for both group and owner.

```
sysadmin@Linux-OS:~$ sudo chmod go-w file2.txt
sysadmin@Linux-OS:~$
```

3. Compare the output of the command `ls -l` for files `file1.txt` and `file2.txt`.

```
-rw-rw-r-- 1 Michael sysadmin 0 Nov 1 12:53 file1.txt
-rw-rw-r-- 1 Michael Michael 0 Nov 1 12:53 file2.txt

-rwxrw-r-- 1 Michael sysadmin 0 Nov 1 12:53 file1.txt
-rw-r--r-- 1 Michael Michael 0 Nov 1 12:53 file2.txt
```

(before making Permission Changes)

(after making Permission Changes)

3.2 Recursive Permission Changes

1. Type the command `mkdir testdir` to create a new directory.

```
sysadmin@Linux-OS:~$ mkdir testdir
sysadmin@Linux-OS:~$
```

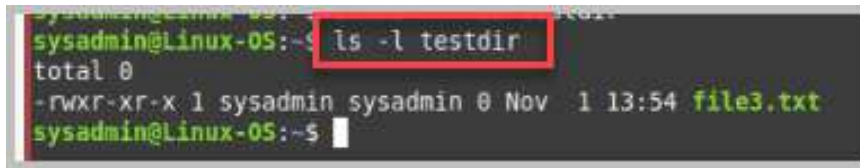
2. Type the command `touch testdir/file3.txt` to create a new file in the `testdir` directory.

```
sysadmin@Linux-OS:~$ touch testdir/file3.txt
sysadmin@Linux-OS:~$
```

3. Type the command `chmod -R 755 testdir` to change permissions for all contents of the `testdir` directory.

```
sysadmin@Linux-OS:~$ chmod -R 755 testdir
sysadmin@Linux-OS:~$
```

4. Type the command `ls -l testdir` to check permissions for the files in the *testdir* directory.



```
sysadmin@Linux-OS:~$ ls -l testdir
total 0
-rwxr-xr-x 1 sysadmin sysadmin 0 Nov  1 13:54 file3.txt
sysadmin@Linux-OS:~$
```



Recursive permission changes in Linux involve altering permissions for a directory and all its contents, including subdirectories and files, using the `-R` option with commands like `chmod` or `chown`. For example, `chmod -R 755 directoryname` sets permissions for `directoryname` and everything within it.

755: This is the permission set in numeric mode:

- 7 for the owner (read, write, execute)
- 5 for the group and others (read, execute)

Directory Access: Everyone can list the contents of `testdir` and navigate into it (`x` permission). **File Access:** Files within `testdir` can be read by everyone but only modified by the owner. This command is commonly used to set standard permissions for web directories, ensuring that files are accessible to web servers while maintaining security by restricting write access to the owner.

5. The lab is now complete; you may end the reservation.